

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
print("hello world")
```

hello world

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) archive (6).zip

archive (6).zip(application/x-zip-compressed) - 35013 bytes, last modified: 5/29/2026 - 100% done
Saving archive (6).zip to archive (6) (1).zip

```
df = pd.read_csv("lahore_aqi_2019_to_2023.csv")
df.head()
```

	date	aqi_pm2.5	max_temp_f	avg_temp_f	min_temp_f	max_dew_point_f	avg_dew_point_f	min_dew_point_f	max_humidity_percent	avg_humidity_percent	min_humi
0	01-06-19	222.0	115	101.0	88	64	56.2	39	40	24.6	
1	02-06-19	170.0	115	100.3	86	64	61.1	57	45	28.6	
2	03-06-19	223.0	109	96.5	84	66	61.4	57	49	33.1	
3	04-06-19	153.0	106	92.1	81	72	64.3	57	52	40.1	
4	05-06-19	157.0	111	94.8	82	68	60.5	52	55	34.1	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
# Display settings
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10,5)
```

```
# Loading AQI Dataset
```

```
import zipfile
```

```
df = pd.read_csv("lahore_aqi_2019_to_2023.csv")
```

```
# Display first 5 rows
```

```
df.head()
```

	date	aqi_pm2.5	max_temp_f	avg_temp_f	min_temp_f	max_dew_point_f	avg_dew_point_f	min_dew_point_f	max_humidity_percent	avg_humidity_percent	min_hum
0	01-06-19	222.0	115	101.0	88	64	56.2	39	40	24.6	
1	02-06-19	170.0	115	100.3	86	64	61.1	57	45	28.6	
2	03-06-19	223.0	109	96.5	84	66	61.4	57	49	33.1	
3	04-06-19	153.0	106	92.1	81	72	64.3	57	52	40.1	
4	05-06-19	157.0	111	94.8	82	68	60.5	52	55	34.1	

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
# Shape of dataset
```

```
df.shape
```

```
(1644, 17)
```

```
# Column information
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1644 entries, 0 to 1643
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                  1644 non-null   object
1   aqi_pm2.5             1423 non-null   float64
2   max_temp_f            1644 non-null   int64
3   avg_temp_f            1644 non-null   float64
4   min_temp_f            1644 non-null   int64
```

```

5 max_dew_point_f      1644 non-null int64
6 avg_dew_point_f      1644 non-null float64
7 min_dew_point_f      1644 non-null int64
8 max_humidity_percent 1644 non-null int64
9 avg_humidity_percent 1644 non-null float64
10 min_humidity_percent 1644 non-null int64
11 max_wind_speed_mph   1644 non-null int64
12 avg_wind_speed_mph   1644 non-null float64
13 min_wind_speed_mph   1644 non-null int64
14 max_pressure_in      1644 non-null float64
15 avg_pressure_in      1644 non-null float64
16 min_pressure_in      1644 non-null float64
dtypes: float64(8), int64(8), object(1)
memory usage: 218.5+ KB

```

```
# Statistical summary
```

```
df.describe()
```

	aqi_pm2.5	max_temp_f	avg_temp_f	min_temp_f	max_dew_point_f	avg_dew_point_f	min_dew_point_f	max_humidity_percent	avg_humidity_percent	min_hu
count	1423.000000	1644.000000	1644.000000	1644.000000	1644.000000	1644.000000	1644.000000	1644.000000	1644.000000	
mean	179.483486	86.406934	76.345195	64.703771	64.896594	60.455535	54.293187	81.675182	61.856083	
std	83.001319	14.330266	13.856784	18.100933	11.976657	11.640406	15.835250	13.330038	14.931803	
min	31.000000	46.000000	40.200000	0.000000	37.000000	31.900000	0.000000	35.000000	20.500000	
25%	124.000000	77.000000	64.850000	54.000000	55.000000	51.100000	45.000000	77.000000	53.600000	
50%	158.000000	90.000000	80.050000	70.000000	64.000000	59.300000	54.000000	84.000000	63.100000	
75%	214.000000	97.000000	87.500000	79.000000	75.000000	72.500000	68.000000	89.000000	71.500000	
max	503.000000	196.000000	102.100000	91.000000	194.000000	81.100000	79.000000	100.000000	98.700000	

```
# Checking missing values
```

```
df.isnull().sum()
```

	0
date	0
aqi_pm2.5	221
max_temp_f	0
avg_temp_f	0
min_temp_f	0
max_dew_point_f	0
avg_dew_point_f	0
min_dew_point_f	0
max_humidity_percent	0
avg_humidity_percent	0
min_humidity_percent	0
max_wind_speed_mph	0
avg_wind_speed_mph	0
min_wind_speed_mph	0
max_pressure_in	0
avg_pressure_in	0
min_pressure_in	0

dtype: int64

```
# data cleaning
```

```
# Removing duplicate records
```

```
df.drop_duplicates(inplace=True)
```

```
df
```

	date	aqi_pm2.5	max_temp_f	avg_temp_f	min_temp_f	max_dew_point_f	avg_dew_point_f	min_dew_point_f	max_humidity_percent	avg_humidity_percent	min_humidity_percent
0	01-06-19	222.0	115	101.0	88	64	56.2	39	40	24.6	
1	02-06-19	170.0	115	100.3	86	64	61.1	57	45	28.6	
2	03-06-19	223.0	109	96.5	84	66	61.4	57	49	33.1	
3	04-06-19	153.0	106	92.1	81	72	64.3	57	52	40.1	
4	05-06-19	157.0	111	94.8	82	68	60.5	52	55	34.1	
...	...	...	...	...	...	...	...	...	...	...	...
1639	26-11-23	365.0	73	62.9	54	54	52.7	50	88	70.5	
1640	27-11-23	378.0	70	62.7	55	55	54.3	52	88	74.4	
1641	28-11-23	358.0	75	64.6	55	61	57.0	52	88	76.9	
1642	29-11-23	332.0	77	67.0	57	59	55.8	54	88	69.4	
1643	30-11-23	225.0	70	63.4	59	61	56.6	55	88	79.6	

1644 rows × 17 columns

Next steps:

[Generate code with df](#)[New interactive sheet](#)

Filling missing numerical values

df.fillna(df.mean(numeric_only=True), inplace=True)

df

	date	aqi_pm2.5	max_temp_f	avg_temp_f	min_temp_f	max_dew_point_f	avg_dew_point_f	min_dew_point_f	max_humidity_percent	avg_humidity_percent	min_humidity_percent
0	01-06-19	222.0	115	101.0	88	64	56.2	39	40	24.6	
1	02-06-19	170.0	115	100.3	86	64	61.1	57	45	28.6	
2	03-06-19	223.0	109	96.5	84	66	61.4	57	49	33.1	
3	04-06-19	153.0	106	92.1	81	72	64.3	57	52	40.1	
4	05-06-19	157.0	111	94.8	82	68	60.5	52	55	34.1	
...	...	...	...	...	...	...	...	...	...	...	...
1639	26-11-23	365.0	73	62.9	54	54	52.7	50	88	70.5	
1640	27-11-23	378.0	70	62.7	55	55	54.3	52	88	74.4	
1641	28-11-23	358.0	75	64.6	55	61	57.0	52	88	76.9	
1642	29-11-23	332.0	77	67.0	57	59	55.8	54	88	69.4	
1643	30-11-23	225.0	70	63.4	59	61	56.6	55	88	79.6	

1644 rows × 17 columns

Next steps:

[Generate code with df](#)[New interactive sheet](#)

coloumn names check

Displaying all column names

df.columns

```
Index(['date', 'aqi_pm2.5', 'max_temp_f', 'avg_temp_f', 'min_temp_f',
      'max_dew_point_f', 'avg_dew_point_f', 'min_dew_point_f',
      'max_humidity_percent', 'avg_humidity_percent', 'min_humidity_percent',
      'max_wind_speed_mph', 'avg_wind_speed_mph', 'min_wind_speed_mph',
      'max_pressure_in', 'avg_pressure_in', 'min_pressure_in'],
      dtype='object')
```

```
# example
pd.Index(['Date', 'AQI', 'PM2.5', 'PM10'], dtype='object')
```

```
Index(['Date', 'AQI', 'PM2.5', 'PM10'], dtype='object')
```

```
# data conversion
```

```
# Converting date column into datetime format
```

```
df['date'] = pd.to_datetime(df['date'])
```

```
/tmp/ipykernel_9572/2450435481.py:3: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure p
df['date'] = pd.to_datetime(df['date'])
```

```
df["date"]
```

```

      date
0  2019-01-06
1  2019-02-06
2  2019-03-06
3  2019-04-06
4  2019-05-06
...      ...
1639 2023-11-26
1640 2023-11-27
1641 2023-11-28
1642 2023-11-29
1643 2023-11-30
1644 rows × 1 columns

dtype: datetime64[ns]
```

```
df['Year'] = df['date'].dt.year
df['Month'] = df['date'].dt.month
```

```
df['Day'] = df['date'].dt.day
```

```
df['Year']
```

	Year
0	2019
1	2019
2	2019
3	2019
4	2019
...	...
1639	2023
1640	2023
1641	2023
1642	2023
1643	2023

1644 rows × 1 columns

dtype: int32

```
df['Month']
```

	Month
--	-------

0	1
---	---

1	2
---	---

2	3
---	---

3	4
---	---

4	5
---	---

...	...
-----	-----

1639	11
------	----

1640	11
------	----

1641	11
------	----

1642	11
------	----

1643	11
------	----

1644 rows × 1 columns

dtype: int32

df['Day']

	Day
--	-----

0	6
---	---

1	6
---	---

2	6
---	---

3	6
---	---

4	6
---	---

...	...
-----	-----

1639	26
------	----

1640	27
------	----

1641	28
------	----

1642	29
------	----

1643	30
------	----

1644 rows × 1 columns

dtype: int32

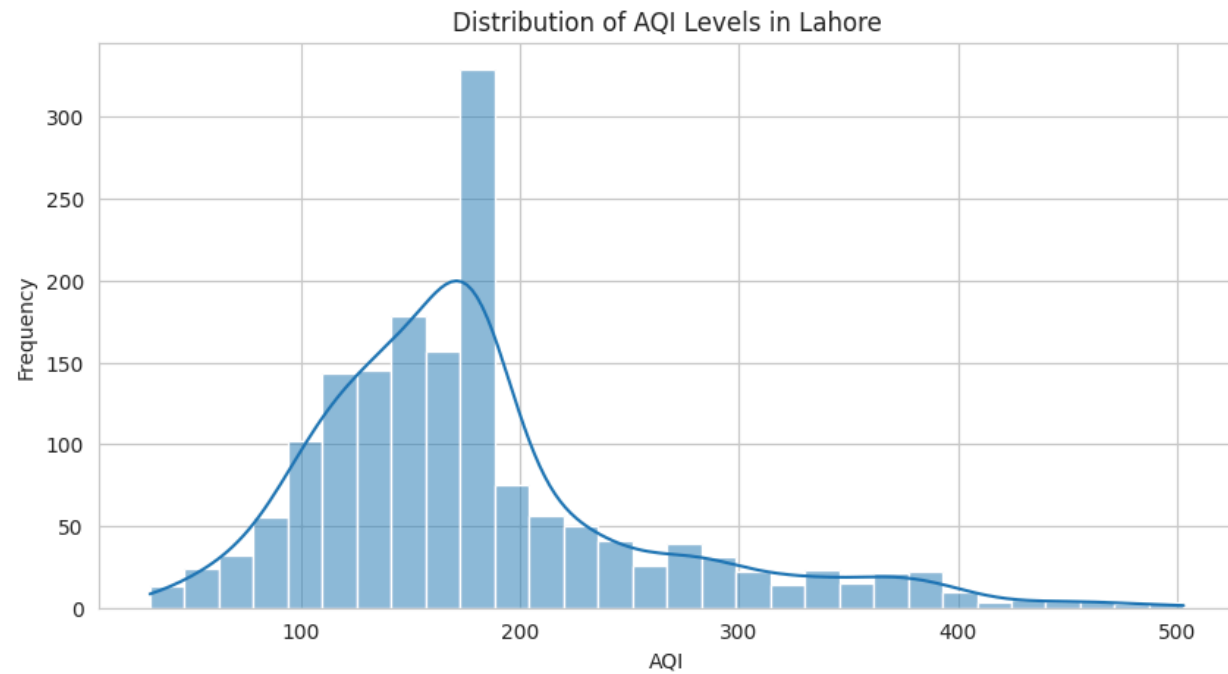
```
# aqi distribution graph
```

```
# AQI Distribution
```

```
plt.figure()  
sns.histplot(df['aqi_pm2.5'], bins=30, kde=True)
```

```
plt.title("Distribution of AQI Levels in Lahore")  
plt.xlabel("AQI")  
plt.ylabel("Frequency")
```

```
plt.show()
```



```
# AQI Trend Analysis

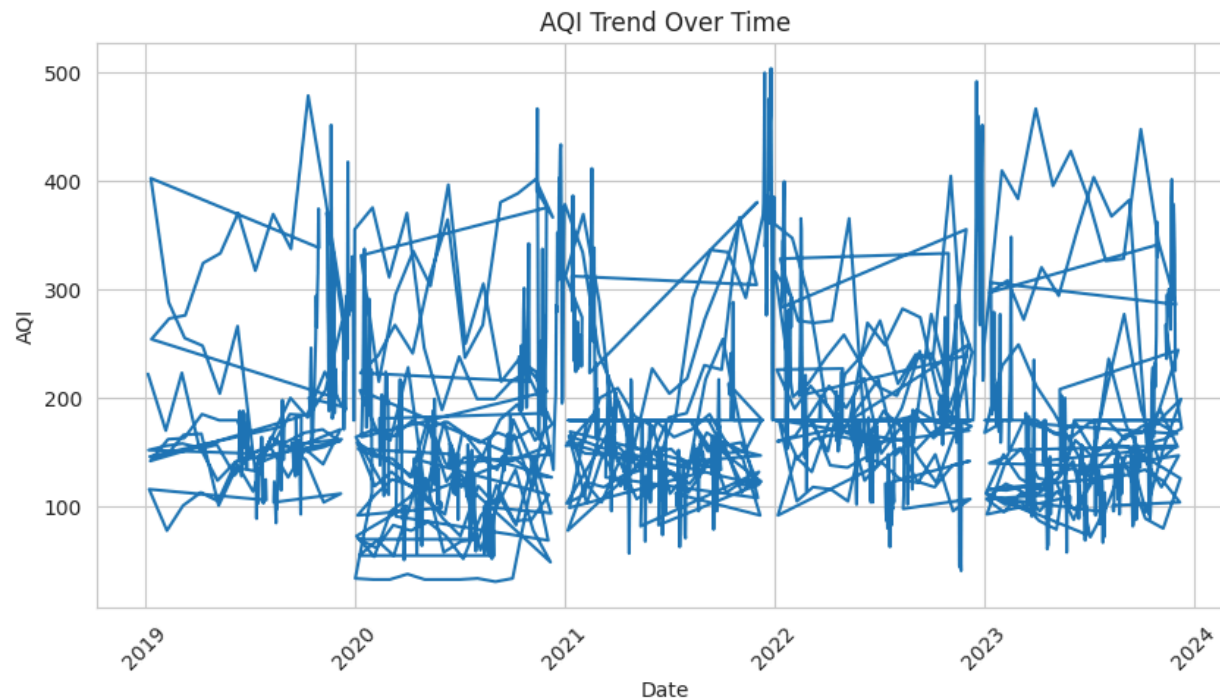
plt.figure()

plt.plot(df['date'], df['aqi_pm2.5'])

plt.title("AQI Trend Over Time")
plt.xlabel("Date")
plt.ylabel("AQI")

plt.xticks(rotation=45)

plt.show()
```



```
#boxplot (outliar)
```

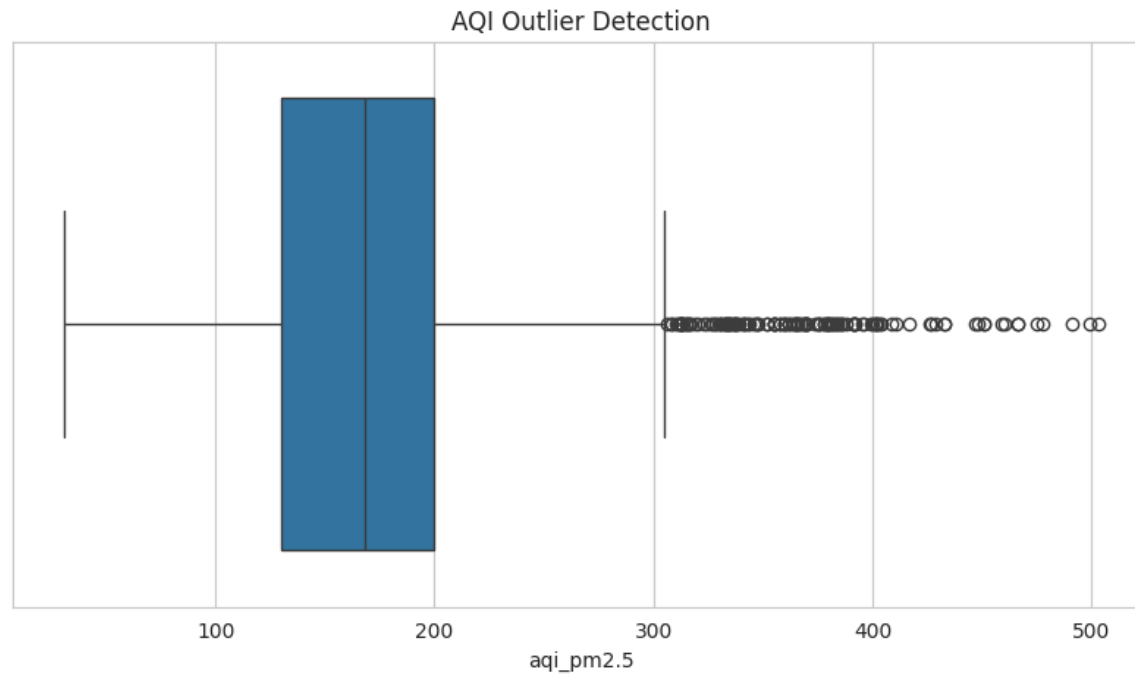
```
# Detecting Outliers in AQI

plt.figure()

sns.boxplot(x=df['aqi_pm2.5'])

plt.title("AQI Outlier Detection")

plt.show()
```



#correlation heat map

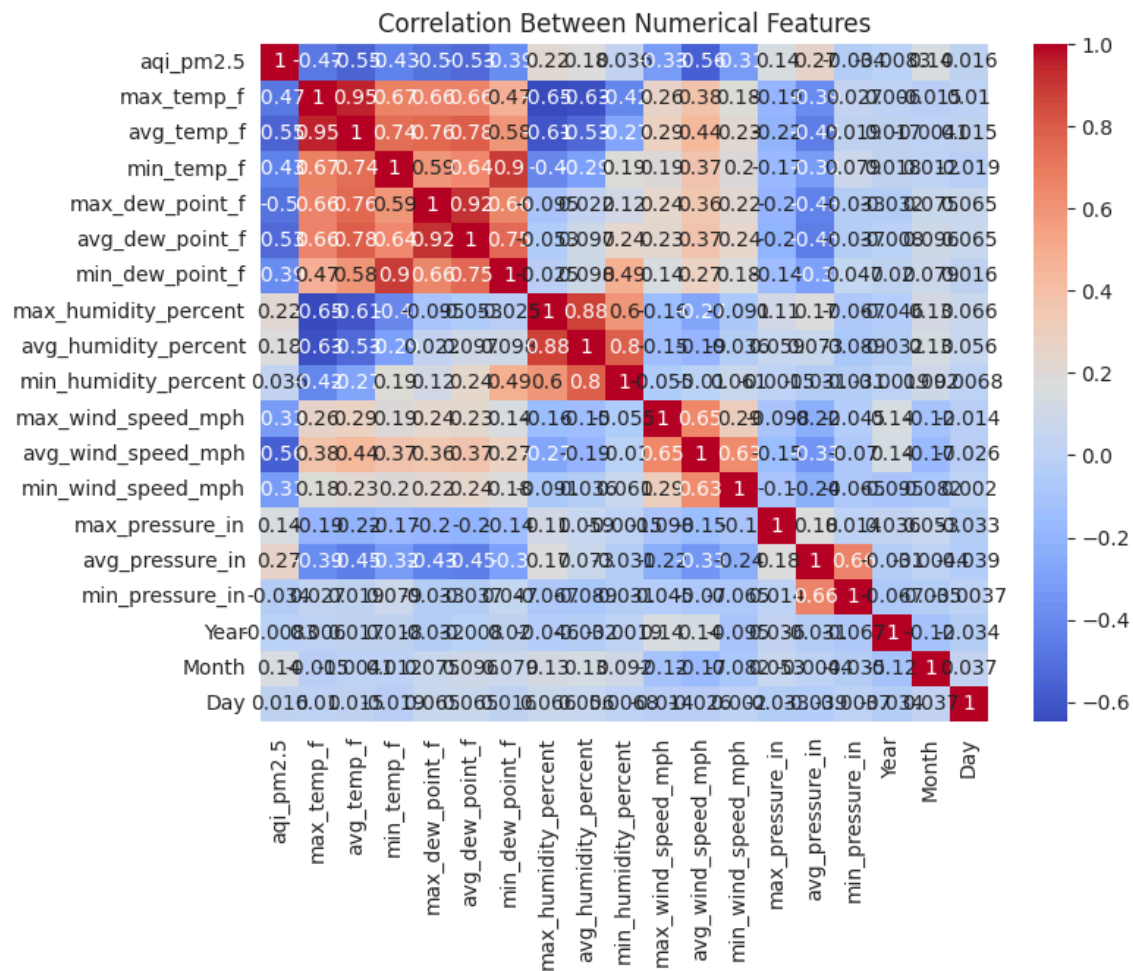
```
# Correlation Heatmap

plt.figure(figsize=(8,6))

sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm')

plt.title("Correlation Between Numerical Features")

plt.show()
```



```
# monthly aqi analysis
```

```
# Monthly Average AQI
```

```
monthly_aqi = df.groupby('Month')['aqi_pm2.5'].mean()
```

```
monthly_aqi
```

aqi_pm2.5	
Month	
1	216.687064
2	195.475939
3	164.846570
4	156.462320
5	153.820790
6	154.738092
7	134.213435
8	145.645323
9	163.094699

```
# Monthly AQI Visualization
```

```
monthly_aqi.plot(kind='bar')
```